

Dinámica y movimiento para videojuegos y gamificación

Tema 3. Dinámica de rotación. Sólido rígido

Pedro Ángel Bernaola Galván

Febrero 2026

Contenido

- 1 Introducción
- 2 Fuerza centrípeta
- 3 Centro de masas
- 4 El Sólido Rígido. Momento de inercia
- 5 Rodadura y deslizamiento
- 6 Orientación en 3D
- 7 Rozamiento por Rodadura

1 Introducción

Dinámica de Rotación: Introducción

En el desarrollo de videojuegos, no basta con que los objetos se desplacen; deben girar, volcar y reaccionar a colisiones excéntricas.

Pasamos del modelo de partícula al de sólido con extensión espacial:

- **Pymunk:** Maneja rotaciones 2D mediante el momento de inercia y el *torque*.
- **PyBullet:** Simula rotaciones en 3D (cuaterniones y matrices de rotación).
- **Objetivo:** Relacionar las fuerzas aplicadas con el cambio en el estado de rotación.

Cinemática Angular

Para describir el giro, definimos magnitudes análogas a las lineales:

Magnitudes fundamentales

- **Espacio angular (θ):** Ángulo recorrido en radianes.
- **Velocidad angular (ω):** Variación del ángulo por unidad de tiempo.

$$\omega = \frac{d\theta}{dt}$$

- **Aceleración angular (α):** Variación de la velocidad angular.

$$\alpha = \frac{d\omega}{dt} = \frac{d^2\theta}{dt^2}$$

Nota: En Pymunk, la velocidad angular se mide en rad/s.

2 Fuerza centrípeta

La Física del Giro: Fuerza Centrípeta

- **Definición:** Es la componente de la fuerza neta que actúa de forma perpendicular a la velocidad instantánea del cuerpo en cada punto del camino.
- Si queremos que un cuerpo que viaje a velocidad v describa una curva de radio R , esta componente tiene que ser igual a:

$$F_c = m \frac{v^2}{R}$$

- **Naturaleza:** Para un coche es el rozamiento con el suelo, para un tren la fuerza que ejerce el raíl, para un avión la fuerza del aire sobre los alerones, para un satélite la gravedad. `gravitacion04.py`

Relevancia en Videojuegos

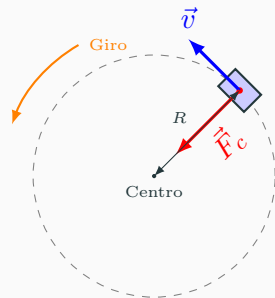
Sin esta fuerza, los objetos seguirían trayectorias rectas por inercia. Es vital para simular el agarre en curvas o las órbitas en simuladores espaciales.

Aplicación en Videojuegos: Steering y Curvas

Para lograr una simulación realista, el motor gestiona el equilibrio entre la inercia lineal del cuerpo y la fuerza lateral que permite realizar el giro.

- **Derrape:** Si la F_c necesaria es mayor que la fricción disponible, el coche pierde el agarre lateral.
- **Cámaras:** Se usa para calcular trayectorias orbitales suaves y fluidas alrededor del jugador.
- **Estabilidad:** Una F_c aplicada muy arriba o muy abajo puede provocar el vuelco del vehículo.

$$\text{Condición de agarre: } m \frac{v^2}{R} \leq F_r$$

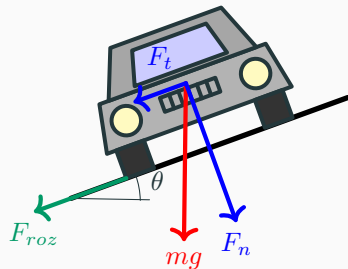


`fcentripeta01.py`
`parabolica_monza.py`

Peralte

- Es la **inclinación transversal** de la calzada en las curvas de una carretera o vía férrea.
- Se define por el ángulo θ respecto a la horizontal.
- Añade una componente del peso a la fuerza centrípeta

$$F_c = F_{roz} + F_t = F_{roz} + mg \sin \theta$$



- **Aumento de velocidad crítica:** Permite que los vehículos tomen curvas a mayores velocidades sin depender exclusivamente del rozamiento neumático-asfalto.
- **Seguridad:** Evita el **deslizamiento** hacia el exterior de la curva en condiciones de baja adherencia (lluvia o hielo), ya que la propia geometría de la vía ayuda a mantener el coche en trayectoria.
- **Confort:** Reduce la sensación de empuje lateral para los pasajeros.

¿Fuerza Centrífuga? El Observador Interno

Desde dentro de un vehículo en una curva, percibimos una fuerza que nos empuja hacia el exterior. ¿Es una fuerza real?

- **La Realidad (Sistema Inercial):** Un observador externo ve que tu cuerpo intenta seguir en línea recta debido a la **Inercia**. Es el coche el que, al girar, “choca” contra ti para obligarte a curvar.
- **La Percepción (Sistema No Inercial):** Si situamos el sistema de referencia dentro del coche (que está acelerando), para que las leyes de Newton sigan funcionando debemos añadir una **Fuerza Ficticia** o de inercia: la **Fuerza Centrífuga**.

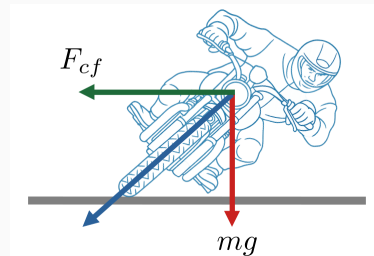
Fuerza Centrífuga ($\vec{F}_{cf}$)

Es una fuerza aparente, de igual magnitud y sentido opuesto a la centrípeta, que solo existe para el observador que gira:

$$F_{cf} = m \frac{v^2}{R}$$

Las curvas en Moto GP: Inclinación y Equilibrio

- Para que un motorista no vuelque en una curva, debe inclinar la moto un ángulo ϕ . Desde su sistema de referencia, busca el **equilibrio de fuerzas**:
- **Condición de Estabilidad** La fuerza **Resultante** ($\vec{R} = \vec{P} + \vec{F}_{cf}$) debe pasar exactamente por la línea que une el CM y el punto de contacto de los neumáticos.
- **Ángulo óptimo:** $\tan(\phi) = \frac{F_{cf}}{mg} = \frac{v^2}{gR}$.
- Si se inclina menos de lo debido para su velocidad, la resultante cae "fuera" del apoyo y la moto tiende a levantarse (subviraje); si inclina demasiado se cae.
- Explicar esto sin fuerzas ficticias es complicado (se puede, por supuesto)



Resultante alineada con el punto de apoyo.

3 Centro de masas

Rotación y centro de masas

La dinámica de rotación para una **masa puntual** no presenta demasiada complicación.

- La componente tangencial de la fuerza puede modificar la velocidad angular.
- La componente normal puede aumentar o disminuir el radio de giro.

Sin embargo, los cuerpos que simulamos no son puntuales, poseen **volumen y distribución de masa**.

- Los hemos tratado como partículas porque hemos aplicado siempre las fuerzas en el **centro de masas**: Punto donde puede considerarse concentrada toda la masa del cuerpo para describir su movimiento traslacional.
- **Centro de gravedad** Punto donde actúa la resultante de las fuerzas gravitatorias sobre el cuerpo.
 - En un campo gravitatorio **uniforme**, ambos puntos coinciden.
 - En general pueden diferir si el campo gravitatorio no es uniforme.

Cálculo del centro de masas

- A partir de ahora supondremos que el centro de masas y el centro de gravedad coinciden.
- **Sistema discreto de masas.** Si el cuerpo está formado por partículas puntuales m_i situadas en $\vec{r}_i$,

$$\vec{r}_{CM} = \frac{1}{M} \sum_i m_i \vec{r}_i, \quad M = \sum_i m_i$$

- **Distribución continua de masa.** Para un cuerpo con densidad $\rho(\vec{r})$,

$$\vec{r}_{CM} = \frac{1}{M} \int \vec{r} \rho(\vec{r}) dV, \quad M = \int \rho(\vec{r}) dV$$

- Es una **posición media ponderada por la masa**.
- Describe el movimiento traslacional del cuerpo como si toda su masa estuviera concentrada en ese punto.

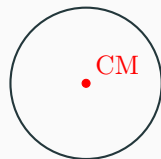
Centro de masas en pymunk

En `pymunk` los cuerpos rígidos se modelan mediante **figuras planas** (2D). Si asumimos **densidad uniforme**, el centro de masas coincide con el **centro geométrico** de la figura cuando este está claramente definido.

- Círculo, cuadrado o rectángulo, elipse, ...
- En estos casos el centro de masas se sitúa en ese punto:

$$\vec{r}_{CM} = \vec{r}_{\text{geom}}$$

- Esto simplifica los cálculos de simulación: el movimiento traslacional se puede tratar como si toda la masa estuviera concentrada en $\vec{r}_{CM}$, y la rotación se describe respecto a este punto.



Círculo



Rectángulo

En `pymunk` el origen de coordenadas dentro del cuerpo siempre se toma en el CM

Centro de masas de un triángulo

Si la **densidad es uniforme** coincide con el baricentro del triángulo

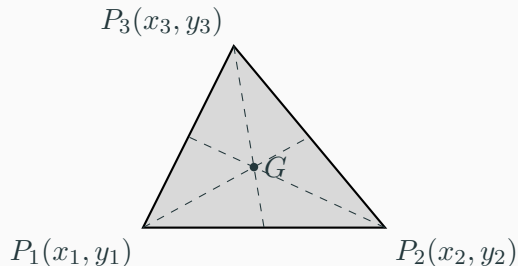
Triángulo de vértices

$P_1(x_1, y_1), P_2(x_2, y_2), P_3(x_3, y_3)$

- El **baricentro** G es el *punto de intersección de las medianas*.
- Se calcula promediando las coordenadas de los vértices:

$$x_{CM} = \frac{x_1 + x_2 + x_3}{3}, \quad y_{CM} = \frac{y_1 + y_2 + y_3}{3}$$

- El baricentro divide cada mediana en proporción 2:1 desde el vértice hasta el punto medio del lado opuesto.



Centro de masas de un polígono convexo

Polígono convexo: Vértices $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ enumerados en orden.

Área del polígono:

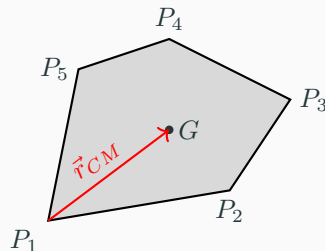
$$A = \frac{1}{2} \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i), \quad P_{n+1} = P_1$$

Centro de masas (centroide):

$$x_{CM} = \frac{1}{6A} \sum_{i=1}^n (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

$$y_{CM} = \frac{1}{6A} \sum_{i=1}^n (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

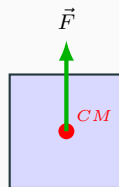
- Fórmulas válidas para cualquier polígono convexo o simple.
- El centroide es el centro de masas si **si el polígono tuviera densidad uniforme**.



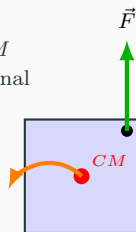
Efecto de la fuerza sobre el centro de masas

La **inercia del cuerpo** actúa sobre el centro de masas:

- Si aplicamos una fuerza sobre el $\vec{r}_{CM}$, el cuerpo se desplaza como una **masa puntual**.
- Si la fuerza se aplica sobre otro punto el cuerpo gira.
- En Pymunk, para aplicar una fuerza en el CM :
`body.apply_force_at_local_point(force, point=(0,0))`
- Para $(x, y) \neq (0,0)$ la fuerza se aplica en el punto de coordenadas (x, y) **con respecto al CM**



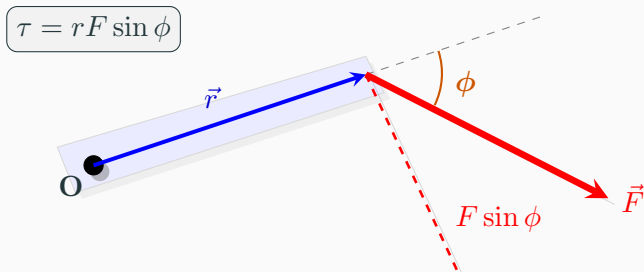
Fuerza sobre el CM
Movimiento traslacional



Fuerza fuera del CM
Traslación + Rotación

Momento de una Fuerza (Torque)

El **torque** ($\vec{\tau}$) representa la capacidad de una fuerza para producir una rotación. Depende de la intensidad de la fuerza, como de su punto de aplicación y del ángulo que forma la fuerza con la línea que une el punto de aplicación y el centro de giro.



Interpretación Física

Solo la componente de $\vec{F}$ perpendicular a $\vec{r}$ ($F \sin \phi$) genera giro. Si la fuerza apunta directamente al origen ($\phi = 180^\circ$), el torque es nulo.

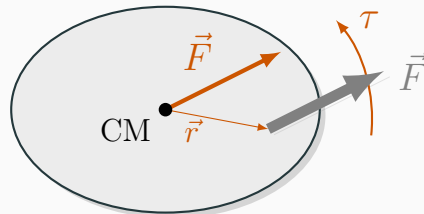
Descomposición: Fuerza neta vs. Torque

Al aplicar una fuerza $\vec{F}$ en un punto $\vec{r}$ alejado del CM , el efecto se descompone en:

- **Traslación:** La fuerza actúa sobre el CM ($\vec{a} = \vec{F}/m$).
- **Rotación:** Se genera un torque neto que cambia la velocidad angular.

- **Principio de Equivalencia**

- $\vec{F}$ excéntrica $\equiv \vec{F}$ en el CM + Torque
- Mayor $|\vec{r}|$, mayor efecto de giro (mayor “brazo de palanca”).



Implementación en Pymunk

`fcentripeta02.py`

Cuando usamos `body.apply_force_at_local_point(force, point)`, Pymunk calcula internamente:

$$\tau = \vec{r} \times \vec{F}$$

4 El Sólido Rígido. Momento de inercia

El Sólido Rígido

Sólido Rígido. Es un sistema de partículas donde las distancias entre ellas permanecen constantes:

$$|\vec{r}_i - \vec{r}_j| = \text{constante}, \quad \forall i, j$$

- **Consecuencia:** No hay deformaciones. El objeto mantiene su forma intacta tras una colisión.
- **Estado:** El movimiento total se descompone en la **traslación del CM** y la **rotación respecto al CM**.
- **Simplicidad:** En Pymunk, esto nos permite definir una única velocidad angular **body.angular_velocity** para todo el cuerpo.

La Resistencia al Giro: Momento de Inercia (I)

Si la masa es la oposición al cambio de movimiento lineal ($F = ma$), el **momento de inercia** es la oposición al cambio de rotación.

Definición para un sólido rígido Depende de cómo se distribuye la masa respecto al eje de giro:

$$I = \sum m_i r_i^2 \simeq \int r^2 dm$$

- **Distribución:** A igual masa, un objeto con la masa alejada del centro (ej. un aro) cuesta más de hacer girar que uno concentrado (ej. un disco).
- **Unidades:** $[I] = \text{kg} \cdot \text{m}^2$ (En **pymunk** $\text{kg} \cdot \text{px}^2$)
- **Ejemplo:** Un una rueda tarda más en caer por un plano inclinado que una esfera **aro_esfera.py**. Sin embargo, si suben una pendiente con la misma velocidad inicial, el aro tarda más en pararse **aro_esfera_subida.py**

Segunda Ley de Newton para la Rotación

Relacionamos la causa del giro (**Torque**) con el efecto cinemático (**Aceleración angular**) a través del **Momento de Inercia** (I).

Ecuación Fundamental

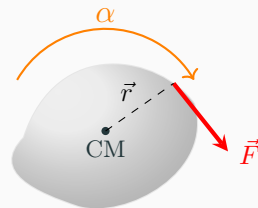
Análoga a $F = ma$ para la traslación:

$$\sum \tau = I\alpha$$

- **Causa** (τ): Torque neto sobre el sólido.
- **Oposición** (I): Resistencia al cambio de rotación.
- **Efecto** (α): Cambio en la velocidad angular ω .

En el motor de física

El motor resuelve en cada *step*: $\alpha = \frac{\sum \tau}{I}$. Integrando para actualizar ω y el ángulo θ .



El torque $\vec{\tau} = \vec{r} \times \vec{F}$ genera la aceleración α .

Analogía: Traslación vs. Rotación

Para facilitar la implementación, podemos ver la rotación como el “reflejo” angular de la traslación:

Magnitud	Lineal	Angular
Causa	Fuerza (F)	Torque (τ)
Oposición	Masa (m)	Inercia (I)
Estado	Velocidad (v)	Vel. angular (ω)
Cambio	Aceleración (a)	Acel. angular (α)
Ley de Newton	$F = ma$	$\tau = I\alpha$

Nota: En Pymunk, `body.torque` acumula todas las fuerzas excéntricas aplicadas antes de cada paso de simulación.

Momentos de Inercia de Figuras Comunes

Figura		Fórmula
Anillo delgado (aro, cilindro hueco)		mR^2
Arandela, cilindro con espesor		$\frac{1}{2}m(R_1^2 + R_2^2)$
Cilindro macizo (disco)		$\frac{1}{2}mR^2$
Esfera sólida		$\frac{2}{5}mR^2$
Corteza esférica		$\frac{2}{5}m \frac{R_2^5 - R_1^5}{R_2^3 - R_1^3}$
Esfera hueca		$\frac{2}{3}mR^2$

Figura		Fórmula
Anillo (Eje diámetro)		$\frac{1}{2}mR^2$
Disco (Eje diámetro)		$\frac{1}{4}mR^2$
Cilindro (Eje central $\perp$)		$\frac{1}{4}mR^2 + \frac{1}{12}mL^2$
Varilla (Eje central $\perp$)		$\frac{1}{12}mL^2$
Placa Rectangular (Eje central $\perp$)		$\frac{1}{12}m(a^2 + b^2)$

Teorema de Steiner (Ejes Paralelos)

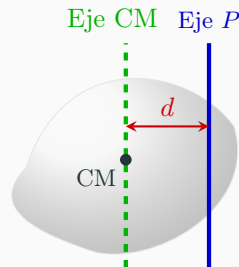
Este teorema permite calcular el momento de inercia I de un sólido rígido respecto a un eje cualquiera, siempre que sea paralelo a un eje que pase por su centro de masas (I_{cm}).

$$I = I_{cm} + md^2$$

- I_{cm} : Momento de inercia respecto al eje del centro de masas.
- m : Masa total del cuerpo.
- d : Distancia perpendicular entre ejes.

Utilidad

Fundamental para sistemas donde el eje de giro está desplazado, como en péndulos físicos o manivelas. Siempre se cumple $I > I_{cm}$.



El momento de inercia aumenta con el cuadrado de la distancia al CM.

Cálculo de Inercia en Pymunk (2D)

Pymunk utiliza fórmulas analíticas para calcular el momento de inercia escalar. Se basa en funciones auxiliares que integran sobre la masa del objeto.

```
1 mass = 1.0
2 # Inercia para un circulo solido
3 inertia_circ = pymunk.moment_for_circle(
4     mass,0,40)
5 # Inercia para un chasis rectangular
6 inertia_box = pymunk.moment_for_box(
7     mass, (160, 40))
8 # Inercia para un poligono convexo (ej.
9     triangulo)
10 # Los vertices se definen relativos al
11     centro de masas
12 vertices = [(-20, -20), (20, -20), (0, 30)]
inertia_poly = pymunk.moment_for_poly(mass,
    vertices,offset=(0,0),radius=0)
```

- La inercia es un único valor float (I_z).
- Los vértices deben definirse **secuencialmente** en sentido horario o antihorario.
- **Convexidad:** Solo es válido para polígonos convexos.
- **offset** sirve para calcular I respecto de un punto desplazado (Utiliza T. de Steiner).
- **radius** para tener en cuenta esquinas redondeadas.

Relación entre Body, Shape e Inercia

En Pymunk, el **body** no tiene dimensiones: es un punto con masa y momento de inercia. La geometría la define el **shape**.

- **Círculos y Cajas:** El **body** se sitúa por defecto en el centro de la figura. El momento de inercia se calcula automáticamente respecto a ese centro.
- **Polígonos:** El **body** se ubica en el punto (0,0) desde el que se han dado las coordenadas.

Ejemplo: Si definimos un triángulo como [(0,0), (10,0), (5,10)], el eje de giro será (0,0) y no su centro de masas (CM). **¿Cómo se corrige?**

```

1  vertices = [(0,0), (10,0), (5,10)]
2  # Forma compacta de corregir el posicionamiento:
3  centroide = pymunk.Poly(pymunk.Body(), vertices).center_of_gravity
4  vertices = [p - centroide for p in vertices]
5  # Ahora la inercia se calcula respecto al CM real
6  inercia = pymunk.moment_for_poly(masa, vertices)
7  cuerpo = pymunk.Body(masa, inercia)
8  forma = pymunk.Poly(cuerpo, vertices)

```

`inercia_rectangulo.py`

Definición de Inercia en PyBullet (URDF)

En PyBullet, el tensor de inercia se define explícitamente en el archivo URDF. Si no se especifica, el motor intenta calcularlo a partir de la geometría de colisión.

```
1  <link name="chasis">
2  <inertial>
3  <origin xyz="0 0 0" rpy="0 0 0"/>
4  <mass value="5.0"/>
5  <inertia ixx="0.1" ixy="0" ixz="0"
6  iyy="0.2" iyz="0"
7  izz="0.2"/>
8  </inertial>
9  </link>
10
```

- Los valores **ixx**, **iiy**, **izz** son los momentos principales.
- Los valores **ixy**, **ixz**, **iyz** son los productos de inercia.

Representación del Tensor de Inercia

PyBullet utiliza una matriz simétrica 3×3 para representar cómo se distribuye la masa en el espacio tridimensional:

$$\mathbf{I} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}$$

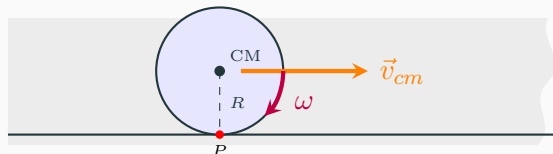
- **Diagonalización:** PyBullet diagonaliza esta matriz para encontrar los ejes principales.
- **Estabilidad:** Valores de inercia incorrectos o nulos pueden causar que el integrador numérico explote (simulación inestable).

5 Rodadura y deslizamiento

Cinemática en plano horizontal: Rodadura Pura

Se dice que un cilindro de radio R realiza una **rodadura pura** cuando el punto de contacto con el suelo tiene velocidad instantánea nula ($\vec{v}_p = 0$).

Condición de rodadura pura: $v_{cm} = \omega R$



- El centro de masas avanza a una velocidad v_{cm} .
- El cuerpo rota con velocidad angular ω .
- En rodadura pura, la distancia recorrida por el CM coincide con la longitud de arco desenrollada: $s = \theta R$.

Estados de deslizamiento y rotación

Cuando la relación $v_{cm} = \omega R$ no se cumple, el cuerpo experimenta deslizamiento. Podemos distinguir tres casos críticos:

Estado	Relación	Efecto visual
Deslizamiento puro	$\omega = 0, v > 0$	El cuerpo “bloqueado” desliza.
Rotación pura (Patina)	$v = 0, \omega > 0$	Rueda girando en el sitio.
Rodadura con deslizamiento	$v \neq \omega R$	El punto P tiene velocidad relativa.

La velocidad del punto de contacto es $\vec{v}_p = \vec{v}_{cm} + \vec{\omega} \times \vec{r}_{p/cm}$. En magnitud, para el punto más bajo:

$$v_p = v_{cm} - \omega R$$

Importante: cuando hay rodadura pura la fuerza de rozamiento solo produce la rotación, **no disipa energía** puesto que no hay movimiento.

¿Cuándo “patina” la rueda?

El término “patinar” o “derrapar” ocurre cuando hay una discrepancia entre la tracción y el avance:

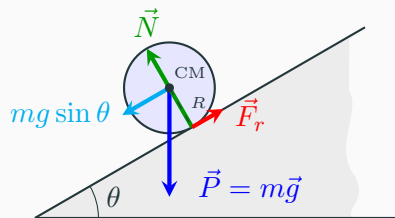
- **Deslizamiento por exceso de velocidad** ($v_{cm} > \omega R$): Ocurre en una frenada brusca con las ruedas bloqueadas. El punto de contacto se arrastra hacia adelante respecto al suelo. [bola_bolos.py](#)
- **Patinaje por exceso de torque** ($v_{cm} < \omega R$): Ocurre al acelerar violentamente sobre una superficie de baja fricción (hielo o barro). La rueda gira más rápido de lo que el vehículo avanza; el punto de contacto se desplaza hacia atrás respecto al suelo. [rueda02.py](#), [coche01.py](#)

La Fuerza de Rozamiento F_r

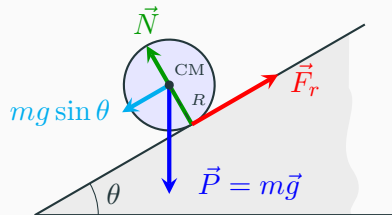
En rodadura pura, la F_r es estática. En el momento en que patina, la F_r pasa a ser cinética ($\mu_k N$), lo que reduce drásticamente el control sobre el movimiento.

Dinámica de un cilindro en plano inclinado

Para un cilindro de masa m y radio R que rueda sin deslizar en un plano de inclinación θ :



Deslizamiento



Rodadura pura

- **Componente tangencial** ($mg \sin \theta$): impulsa el movimiento.
- **Normal** ($\vec{N}$): Reacción del plano ante $mg \cos \theta$.
- **Fuerza de rozamiento** ($\vec{F}_r$): Estática, genera el torque $\tau = F_r R$.

Ecuaciones de Newton-Euler con F_r

El sistema se resuelve vinculando el movimiento lineal y el angular:

$$Mg \sin \theta - F_r = Ma_{cm}$$

$$\tau = F_r R = I\alpha$$

Bajo **rodadura pura** ($a_{cm} = \alpha R$), la aceleración general es:

$$a_{cm} = \frac{g \sin \theta}{1 + \frac{I}{MR^2}}$$

La fricción estática no realiza trabajo ($v_P = 0$). **La energía mecánica se conserva**, transformándose de potencial a cinética (traslación + rotación).

Conclusión: A menor momento de inercia tenemos la masa más concentrada en el centro, cuesta menos hacerla girar por lo que tenemos **mayor aceleración lineal** (que está ligada a la angular). [aro_esfera.py](#), [aro_esfera_subida.py](#).

Aceleración según el cuerpo:

- **Esfera Sólida** ($I = \frac{2}{5}MR^2$):

$$a_{cm} = \frac{5}{7}g \sin \theta \simeq 0,71g \sin \theta$$

- **Cilindro Sólido** ($I = \frac{1}{2}MR^2$):

$$a_{cm} = \frac{2}{3}g \sin \theta \simeq 0,67g \sin \theta$$

- **Cilindro Hueco** ($I = MR^2$):

$$a_{cm} = \frac{1}{2}g \sin \theta = 0,50g \sin \theta$$

Snippet: Aplicando Torque en Pymunk

Para hacer que un objeto rote en un simulador 2D:

```
1  import pymunk
2  # Crear cuerpo y forma
3  # El segundo parametro es el momento de inercia
4  body = pymunk.Body(mass=1, moment=10)
5  # Aplicar torque directamente (en Newton-unidades)
6  body.torque = 500
7  # O aplicar una fuerza excentrica
8  # Fuerza de 100 en el eje Y, aplicada a 20 unidades del centro
9  # Esto genera un torque automatico: tau = r * F
10 body.apply_force_at_local_point((0, 100), (20, 0))
11
```

Recuerda: El momento de inercia es la “resistencia” a este torque. En Pymunk, si el cuerpo es estático, el momento de inercia debe ser infinito (`float('inf')`).

6 Orientación en 3D

Ángulos de Euler (convenio de Tait-Bryan)

- Varias convenciones. En el tema 1 vimos la que se utiliza en Física Teórica ($Z-X-Z$).
- La más usada en Aviación, Robótica y PyBullet es el **convenio de Tait-Bryan**.

Convención Aeroespacial (XYZ)

- **Roll (Alabeo):** Eje X Longitudinal
- **Pitch (Cabeceo):** Eje Y Transv.
- **Yaw (Guiñada):** Eje Z Vertical

Limitaciones Críticas

- **Orden:** El resultado varía según la secuencia (XYZ , ZYX , etc.).
- **Gimbal Lock:** o Bloqueo de Cardán.
- **Importante:** Las rotaciones son **intrínsecas** (alrededor de los ejes móviles del objeto, no los ejes fijos del mundo).
- **La orientación total** es el producto de las matrices elementales (orden $X \rightarrow Y \rightarrow Z$):

$$R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$$
- PyBullet no usa **Ángulos de Euler** internamente; son solo una interfaz humana. Utiliza **cuaterniones** ($[x, y, z, w]$) para evitar singularidades matemáticas.

¿Por qué evitar los Ángulos de Euler?

Aunque son intuitivos (Roll, Pitch y Yaw), presentan un fallo crítico en simulación física y animación: el **Gimbal Lock** (Bloqueo de cardán).

- **El Problema:** Al rotar un eje 90° , dos de los tres ejes de rotación pueden alinearse.
- **La Consecuencia:** Se pierde un grado de libertad. El motor de física no sabe cómo calcular la rotación intermedia y el objeto da “saltos” o gira erráticamente.
- **La Solución:** Usar una representación que no dependa de rotaciones sucesivas en ejes fijos.

El Bloqueo de Cardán (Singularidad Matemática)

La orientación total es el producto de las matrices elementales: $R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$

1. Matriz Genérica

$$R = \begin{pmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{pmatrix}$$

2. Singularidad ($\theta = 90^\circ$)

$$R_{90^\circ} = \begin{pmatrix} 0 & \sin(\phi - \psi) & \cos(\phi - \psi) \\ 0 & \cos(\phi - \psi) & -\sin(\phi - \psi) \\ -1 & 0 & 0 \end{pmatrix}$$

- **Pérdida de un grado de libertad:** La matriz colapsada ahora solo depende de un parámetro: la diferencia $(\phi - \psi)$.
- **Ambigüedad:** Existen infinitas combinaciones de Roll y Yaw para la misma orientación física.
- **Consecuencia:** El software no puede interpolar trayectorias de forma única, provocando giros bruscos o bloqueos del objeto.

Más allá de Euler: Los Cuaterniones

- Aunque los ángulos de Euler son intuitivos, presentan el problema del “gimbal lock”. Para evitarlo, motores como PyBullet operan internamente con **cuaterniones**.
- Son vectores de 4D, $q = [x, y, z, w]$ cuyas componentes se obtienen a partir de:
 - Un vector unitario $\hat{u}$ que representa un eje de giro.
 - Un ángulo de giro θ

$$q = \left[u_x \sin \left(\frac{\theta}{2} \right), u_y \sin \left(\frac{\theta}{2} \right), u_z \sin \left(\frac{\theta}{2} \right), \cos \left(\frac{\theta}{2} \right) \right]$$

- **Sin singularidades:** Un único giro, no hay bloqueo de cardán.
- **Interpolación (SLERP):** Permiten transiciones suaves entre orientaciones, ideal para el movimiento del balón.
- **Normalización:** El cuaternión debe ser unitario: $\sqrt{x^2 + y^2 + z^2 + w^2} = 1$.

Uso de Cuaterniones en PyBullet

No es necesario calcular los 4 números a mano; el motor ofrece herramientas de traducción.

```
1  import pybullet as p
2  # 1. Definimos lo que entendemos (Euler en radianes)
3  euler = [0, 0, 3.14/4]
4  # 2. Traducimos al lenguaje del motor (Cuaternion)
5  # Retorna una lista [x, y, z, w]
6  quat = p.getQuaternionFromEuler(euler)
7  # 3. Aplicamos la orientacion estable
8  p.resetBasePositionAndOrientation(body_id, [0,0,0], quat)
9
```

Nota sobre el formato

PyBullet almacena el componente escalar w al final: $[x, y, z, w]$. Ten cuidado si importas datos de otros motores (como Unity) que pueden usar $[w, x, y, z]$.

Uso de cuaterniones en PyBullet

En PyBullet, casi todas las funciones de posición y orientación devuelven o aceptan cuaterniones en formato de lista de 4 elementos: `[x, y, z, w]`.

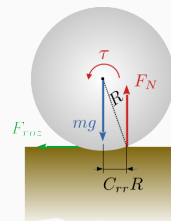
```
1  import pybullet as p
2  # Obtener posicion y orientacion actual
3  pos, quat = p.getBasePositionAndOrientation(body_id)
4  # Convertir de Cuaternion a Matriz de Rotacion (3x3)
5  matrix = p.getMatrixFromQuaternion(quat) # aqui no es necesario
6  # Rotacion relativa: Multiplicar cuaterniones
7  q_rot = p.getQuaternionFromAxisAngle([0, 0, 1], 3.14/4)
8  new_quat = p.multiplyTransforms([0,0,0], quat, [0,0,0], q_rot)[1]
9  # Aplicar la nueva orientacion
10 p.resetBasePositionAndOrientation(body_id, pos, new_quat)
11
```

7 Rozamiento por Rodadura

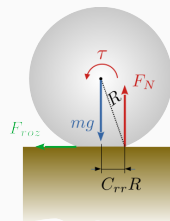
Deformación de los cuerpos que ruedan

A diferencia del rozamiento dinámico, el rozamiento por rodadura aparece debido a la deformación de los cuerpos en contacto.

- Se deforman el suelo, el cuerpo, o ambos.
- La normal ya no se aplica justo debajo del CM y aparece un torque que se opone a la rodadura.
- **Coeficiente de rozamiento por rodadura, C_{rr} :**
 - C_{rr} es adimensional.
 - Se interpreta como la **fracción del radio** que supone la distancia que se aparta de la vertical el punto de aplicación de $\vec{F}_N$.
 - Suele ser mucho menor que el rozamiento por fricción.



Se deforma el suelo



Se deforma el cuerpo

Se genera un **torque de frenado**:

$$\tau = C_{rr} \cdot R \cdot F_N$$

Implementación de Rodadura en Pymunk: Torque Manual

Aplicamos un torque que se opone al giro (ω) y cuya magnitud nunca supera el torque necesario para detener la rueda en un *step*.

```

1    tau_rod = 0.02 * R * (body.mass * 900)    #Crr=0.02 y  FN=mg
2    # 2. Torque para frenado total en un paso (I * w / dt)
3    tau_stop = (body.moment * abs(body.angular_velocity)) / (1/60.0)
4    # 3. Aplicar el minimo para evitar inversion del giro
5    if abs(body.angular_velocity) > 0.01:
6        freno = min(tau_rod, tau_stop)
7        body.torque -= (body.angular_velocity/abs(body.angular_velocity))*freno
8    else:
9        body.angular_velocity = 0 # Detencion limpia
10

```

- El torque es siempre opuesto a ω (siempre **disipativo**).
- $\min(t_{rod}, t_{stop}) \Rightarrow$ nunca puede empujar hacia atrás
- Podemos elegir C_{rr} según la superficie (arena, asfalto, etc.) o tipo de rueda.

Implementación de Rodadura en Pymunk: Torque Manual

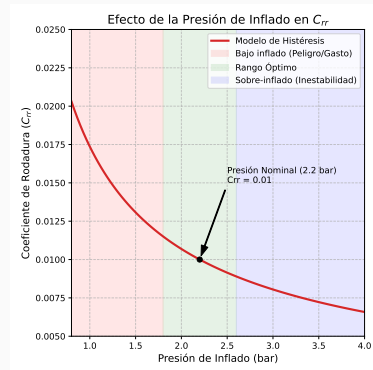
- Le damos al torque resistente el valor máximo $\tau = C_{rr} \cdot R \cdot F_N$
- Pero nunca permitimos que provoque giro hacia atrás.
- C_{rr} lo hemos expresado como una fracción del radio (e.g. 2 %)
- **IMPORTANTE:** Si F_N cambia (e.g. cambia la pendiente o aparece otra fuerza perpendicular a la superficie) hay que modificar a mano el valor del torque
- Sirve también para simular el rozamiento de un eje.

`rueda02.py`

`rueda_roz_rodadura.py`

Influencia de la Presión de Inflado en C_{rr}

- Una **baja presión** aumenta la deformación del neumático (histéresis).
- A mayor flexión de los flancos, mayor disipación de energía en forma de calor.
- $C_{rr} \propto \frac{1}{P^\alpha}$ con α entre 0.5 y 0.9.
- **Asfalto (Suelo duro):** El aumento de presión reduce el C_{rr} de forma asintótica, mejorando la eficiencia hasta alcanzar el límite de rigidez estructural.



- **Compromiso:** Un inflado excesivo reduce el rozamiento pero sacrifica la superficie de contacto (agarre) y la capacidad de absorción de impactos.
- **Impacto Energético:** Una caída del 20 % en la presión puede suponer un incremento del 10 % en la resistencia a la rodadura.

Influencia de la Presión de Inflado en C_{rr}

Rangos Típicos de α (Exponente de Sensibilidad)

- **0.7 – 0.8:** Estándar para neumáticos radiales modernos (turismos).
- **0.5 – 0.6:** Neumáticos de estructura diagonal o alta carga.
- **0.9:** Neumáticos de alta eficiencia o competición.

Interpretación del Valor

- α **Elevado:** Alta sensibilidad al inflado. Pequeñas pérdidas de presión penalizan drásticamente el consumo.
- α **Reducido:** Estructura rígida. El C_{rr} es más estable pero el neumático absorbe peor las irregularidades.
- **Nota sobre el Terreno:** Este modelo asume **suelo rígido**. En suelos deformables (arena/barro), α puede ser negativo (más presión $\rightarrow$ más hundimiento $\rightarrow$ más C_{rr}).

Implementacion en PyBullet (3D)

PyBullet permite definir la rodadura mediante la función `changeDynamics`. Actúa como un torque que modifica el cuaternión (rotación) en cada paso.

```
1  import pybullet as p
2  # Ejemplo con un balon
3  p.changeDynamics(
4      bodyUniqueId=ball_id,
5      linkIndex=-1,
6      lateralFriction=0.5,      # Friccion estandar (deslizamiento)
7      rollingFriction=0.01,    # Resistencia a rodar (e.g. cesp ed)
8      spinningFriction=0.001,  # Resistencia al giro (e.g. rozamiento eje)
9      restitution=0.8 )       # Rebote
10
```

- **rollingFriction**: Implementa el rozamiento por rodadura.
- Este valor se aplica solo durante el **contacto** con otra superficie.

Ejemplo Practico: Cálculo de parámetros

Si tenemos un balón de futbol ($R = 11$ cm) y estimamos una deformación del césped de $b = 1$ mm tendríamos un $C_{rr} = b/R = 0,01$.

- En **Pymunk** no hay que aplicar torque, basta con igualar el parámetro `rollingFriction` al valor de C_{rr} .
- **Torque Físico:** PyBullet aplicará un torque $\tau = C_{rr} \cdot R \cdot F_N$, recalculando F_N si es necesario.

```

1  # Calculo del coeficiente adimensional
2  b = 0.001  # 1 mm
3  R = 0.11   # 11 cm
4  Crr = b / R  # aprox 0.009.
5  p.changeDynamics(ball_id, -1, rollingFriction=Crr)
6

```

Conclusion: Un `rollingFriction` de 0,01 en PyBullet significa que la normal se desplaza un 1 % del radio del objeto hacia adelante.